SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**

Question Count: 5

Code of Conduct

I **Nevitha Magendran/192521206** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Nevitha**

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

   Answer:

   Introduction: A hypervisor is the layer that creates and manages virtual machines. A Type-1 (bare-metal) hypervisor, such as VMware ESXi, Microsoft Hyper-V (server role), Xen or KVM, runs directly on the physical hardware. A Type-2 (hosted) hypervisor, such as VirtualBox or VMware Workstation, runs as an application on top of a host operating system (Windows/Linux/macOS). The comparison for a cloud datacenter hosting mission-critical applications is shown below.

Aspect	Type-1 (Bare-metal)	Type-2 (Hosted)
Performance	Direct access to CPU, memory, NIC and	Every request passes through the host OS

	storage with very low overhead; supports hardware-assisted virtualization (Intel VT-x/AMD-V), SR-IOV, NUMA awareness and large-scale scheduling, giving near-native performance and predictable latency.	and its drivers, adding extra layers, overhead and contention with other host applications; performance is lower and less predictable.
Security	Small attack surface (a thin, purpose-built hypervisor); no general-purpose OS or user applications to exploit; strong VM isolation, secure boot, TPM and signed modules supported.	Large attack surface: a vulnerability or malware in the host OS (or any app on it) can compromise all VMs; patching the host OS affects every VM.
Manageability	Centralized management (vCenter, SCVMM, OpenStack) with live migration, high availability, fault tolerance, resource pools, DRS/auto-scaling, templates, clustering and automation APIs.	Managed individually from the desktop; limited clustering, HA and live-migration features; hard to automate and scale to hundreds of hosts.
Reliability / use case	Designed for 24x7 enterprise and cloud datacenters.	Designed for developers, testing, training and personal use.

Justification: For mission-critical applications the datacenter needs high performance, strong isolation, high availability and centralized automation. Because Type-1 hypervisors provide near-native performance, a smaller attack surface and enterprise features such as live migration, clustering and automatic failover, a Type-1 hypervisor is clearly the better choice. Type-2 would only be suitable for testing or development environments. This is why public clouds (AWS Nitro/KVM, Azure Hyper-V, Google KVM) are all built on Type-1 hypervisors.

2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Answer:

How virtualization supports isolation: Finance, healthcare and student-portal workloads are placed in separate virtual machines (or containers inside VMs) on the same physical host. The hypervisor sits between the VMs and the hardware and enforces isolation in the following ways:

1. Compute isolation – each VM has its own virtual CPUs, guest OS and address space; hardware-assisted virtualization (VT-x/AMD-V, Extended/Nested Page Tables) stops one VM from reading another VM's memory, and the scheduler enforces CPU shares and limits.

2. Memory and storage isolation – memory is allocated per VM with ballooning/page-sharing controlled by the hypervisor, and each VM uses its own virtual disk, encrypted with separate keys.

3. Network isolation – separate virtual switches, VLAN/VXLAN overlays, virtual firewalls and micro-segmentation keep finance, healthcare and student traffic apart.

4. Management and policy isolation – separate tenants/resource pools, role-based access control, per-tenant quotas, and dedicated hosts or anti-affinity rules for the most sensitive tenants (e.g., healthcare/HIPAA and finance/PCI-DSS) so that they never share a host with less trusted workloads. Confidential computing (AMD SEV / Intel TDX) can additionally encrypt VM memory.

Two major attack surfaces:

(i) The hypervisor and virtual-machine escape – a flaw in the hypervisor, device emulation (virtual NIC/USB/graphics drivers) or the management interface may let a malicious VM break out (VM escape) and gain control of the host and every other VM, exposing finance and healthcare data. Mitigation: timely patching, minimal hypervisor, hardened and isolated management network, secure boot, restricted device emulation.

(ii) Shared resources and side channels / virtual network – VMs on the same hardware share CPU caches, memory bus and virtual switches, so side-channel attacks (e.g., Spectre/Meltdown, cache timing), VM-to-VM traffic sniffing through a misconfigured virtual switch, and VM sprawl or insecure snapshots/images can leak data between tenants. Mitigation: dedicated hosts or core pinning for sensitive tenants, micro-segmentation, encrypted traffic, microcode patches, and secured image/snapshot management.

3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Answer:

Analysis of the snapshot: The host is running close to saturation on several resources at once (a healthy host normally keeps CPU below about 70–80%, memory below about 80–85%, storage latency under about 10–20 ms, and boots a VM in well under a minute).

Metric	Observation	Likely bottleneck / cause	Corrective action
CPU 92%	Very high; little headroom for peaks	CPU over-commitment (too many vCPUs per physical core), high CPU-ready/steal time, oversized VMs, noisy-neighbour VMs	Right-size vCPUs, reduce the vCPU:pCore ratio, rebalance with DRS/live migration, add hosts or auto-scale out, set CPU shares/limits
Memory 88%	High; risk of ballooning, swapping and compression	Memory over-commitment, VMs allocated more RAM than needed, memory leaks	Right-size VM memory, enable ballooning/TPS carefully, add RAM or hosts,

			migrate VMs, reserve memory for critical VMs
Storage I/O latency 35 ms	Slow disk response	Shared datastore contention, HDD or congested SAN/network, too many VMs per LUN, boot storms, swapping from memory pressure	Move to SSD/NVMe or tiered storage, spread VMs across datastores, use storage QoS/IOPS limits, caching, faster storage network (multipath, 25/100 GbE)
VM boot time 170 s	Very slow (normal is about 30–60 s)	Combined result of the CPU, memory and storage pressure above, plus large images and many VMs booting at once	Use thin/linked clones and optimized golden images, stagger boots, pre-warm instances, boot from fast storage, lighter OS/containers

Explanation of bottlenecks: The main bottleneck is the storage I/O path together with CPU and memory over-commitment. When memory is nearly full the hypervisor starts ballooning and swapping to disk, which increases I/O on an already slow datastore (35 ms). Slow disks and a busy CPU make operating-system loading and service start-up slow, which explains the 170-second boot time. The problems therefore reinforce each other: high memory causes more disk I/O, and high I/O wait keeps CPUs busy, so fixing only one resource will not be enough.

Overall recommendations: (1) rebalance and migrate VMs to underused hosts using DRS/live migration; (2) right-size VMs and remove idle ones; (3) upgrade to SSD/NVMe storage and apply storage QoS; (4) add capacity or enable auto-scaling to bring CPU below about 75% and memory below about 80%; (5) optimize images and stagger VM start-up; (6) set up continuous monitoring and alerts (CPU-ready, ballooning, datastore latency) with capacity planning to prevent recurrence.

4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.

Answer:

Introduction: A Software Defined Datacenter (SDDC) virtualizes and abstracts all infrastructure — compute, storage and networking — and manages it by software through policies, APIs and automation, instead of by manually configuring dedicated hardware. A traditional datacenter uses hardware-centric silos configured by hand, often with separate teams for servers, storage and networks.

Dimension	**Traditional datacenter**	**Software Defined Datacenter**
Compute	Physical servers dedicated to applications; provisioning takes days or weeks; low	Hypervisors/containers pool CPU and memory; a VM is provisioned in minutes

	utilization (10–20%).	from templates; live migration and auto-scaling give 60–80% utilization.
Storage	Separate SAN/NAS arrays managed per vendor; manual LUN allocation; difficult scaling.	Software-defined storage (vSAN, Ceph) pools local disks; policy-based provisioning, thin provisioning, tiering and replication; scales out by adding nodes.
Network	Hardware switches/routers configured box by box; VLAN changes take hours or days.	Software-defined networking (NSX, OpenFlow) with a central controller, virtual switches/VXLAN overlays, micro-segmentation and virtual firewalls; changes are made by API in seconds.
Management	Manual, ticket-based, error-prone, vendor lock-in.	Automated and policy-driven through orchestration (vRealize, OpenStack, Terraform); self-service portal; consistent templates.

How SDDC improves agility: (1) Speed – entire environments (VMs, storage and networks) can be deployed in minutes through templates and Infrastructure-as-Code, compared with weeks for hardware procurement. (2) Elasticity – resources are pooled and can be scaled up or down on demand, supporting DevOps and CI/CD. (3) Automation – workflows, self-service and policy enforcement reduce manual effort and configuration errors. (4) Flexibility – hardware independence lets the organization use commodity hardware and change vendors easily, and workloads can move between sites or clouds (hybrid cloud). (5) Resilience and security – automated failover, snapshots, and micro-segmentation applied to each workload improve recovery time and security consistency. (6) Cost – higher utilization and lower operational expense compared with over-provisioned, siloed hardware.

Conclusion: By virtualizing compute, storage and network and controlling them through a single software layer, an SDDC lets IT respond to business needs quickly, whereas a traditional datacenter is slow, rigid and costly to change.

5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Answer:

Analysis of the issue: Each cloud provider (AWS, Azure, Google Cloud, private cloud) has its own identity system, user-ID formats, attribute names, roles/permissions model and authentication protocols. When the

federation was built without a common identity framework, the same person appeared with different identifiers (e.g., an e-mail in one cloud and an employee ID in another), roles did not map to equivalent permissions, token formats (SAML, OIDC, proprietary) were incompatible, trust relationships and time/clock or certificate settings were inconsistent, and attributes needed for authorization were missing. The result was failed log-ins, duplicated or orphaned accounts, over-privileged access, and difficulty in auditing and de-provisioning. Sharing full identity data across clouds also raised privacy and compliance concerns.

Proposed secure identity and privacy design:

1. Central Identity Provider (IdP) – choose one authoritative IdP (e.g., Azure AD/Okta/Keycloak) as the source of truth for users, with every cloud acting as a Service Provider that trusts it through federation.

2. Standard protocols – use SAML 2.0 or OpenID Connect for single sign-on and OAuth 2.0 for delegated API access; SCIM for automatic user provisioning and de-provisioning across clouds.

3. Common identity schema and attribute mapping – define a unique, immutable global identifier for each user and a standard attribute set (name, department, role, clearance); map federated attributes and groups to each provider's roles (e.g., IAM roles, Azure RBAC) using claims-mapping rules.

4. Trust framework – establish cross-domain trust using signed metadata and certificates with regular key rotation; use short-lived tokens, token signature validation and synchronized time (NTP).

5. Strong authentication and authorization – enforce multi-factor authentication, role-based and attribute-based access control, least privilege, conditional access, and just-in-time privileged access.

6. Privacy protection – share only the minimum attributes needed (data minimization, attribute release policies), use pseudonymous or pairwise identifiers where possible, encrypt tokens and traffic (TLS), obtain user consent, and respect regional laws such as GDPR and data-residency rules.

7. Governance and monitoring – centralized logging and auditing of authentication events, periodic access reviews, automatic revocation when users leave, and testing of the federation with pilot applications before full roll-out.

Result: users sign in once with a single consistent identity, permissions are mapped uniformly across providers, access is auditable and revocable from one place, and personal data exposure is minimized — solving the identity mismatch and making the multi-cloud federation secure and privacy-preserving.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded: